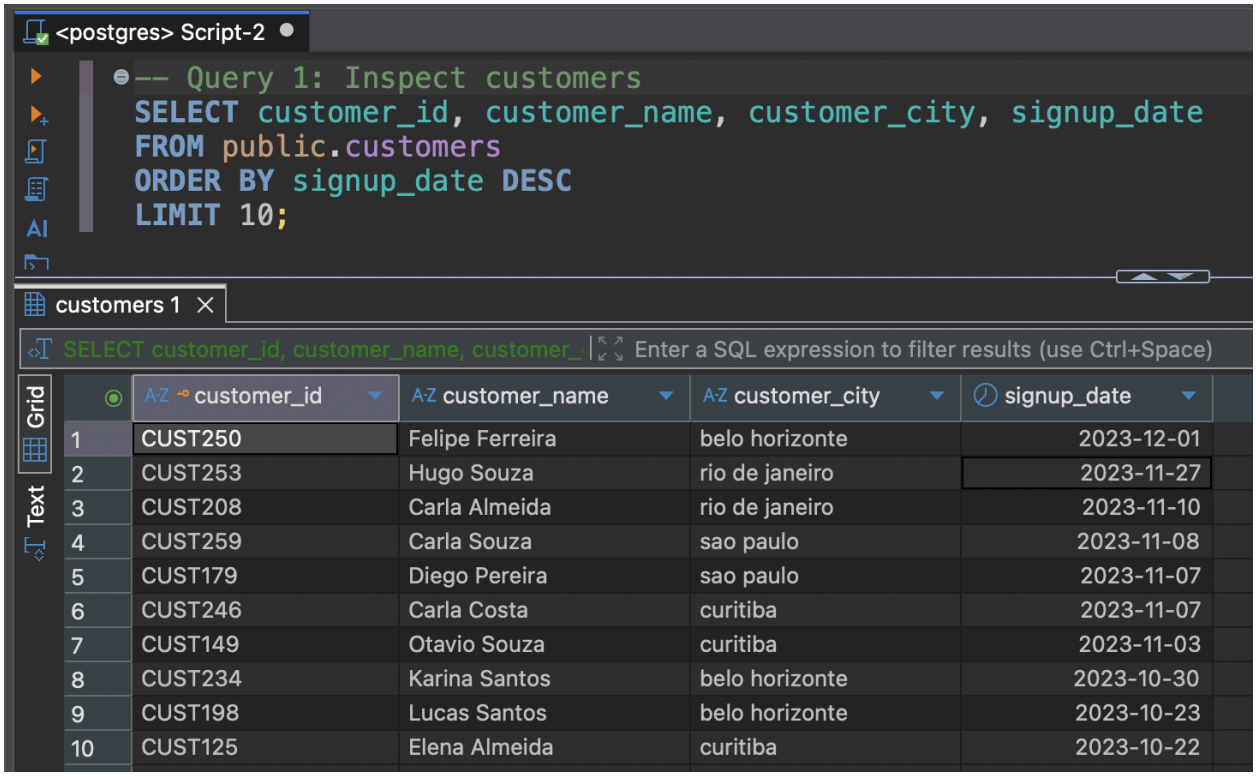
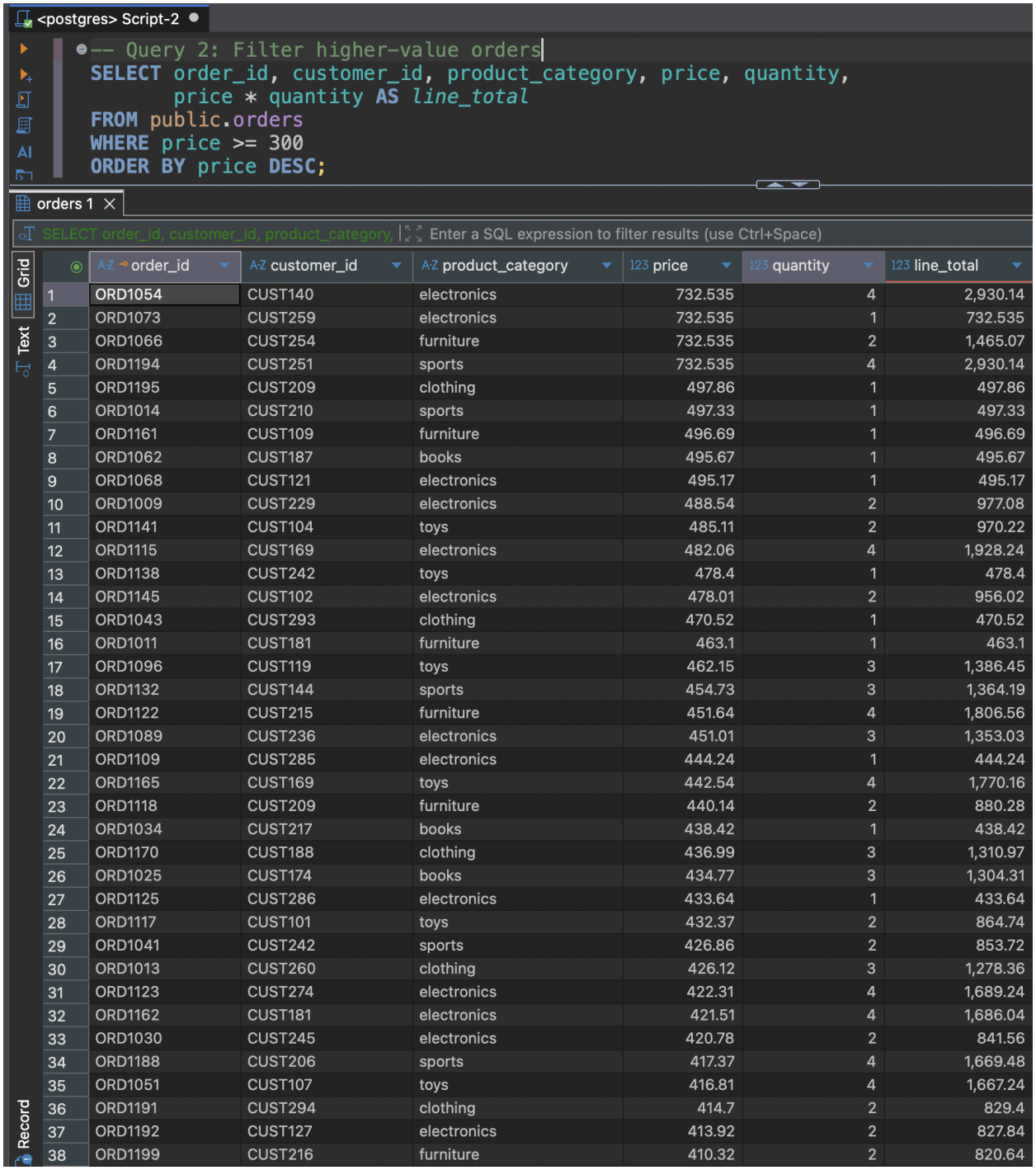
Database 2

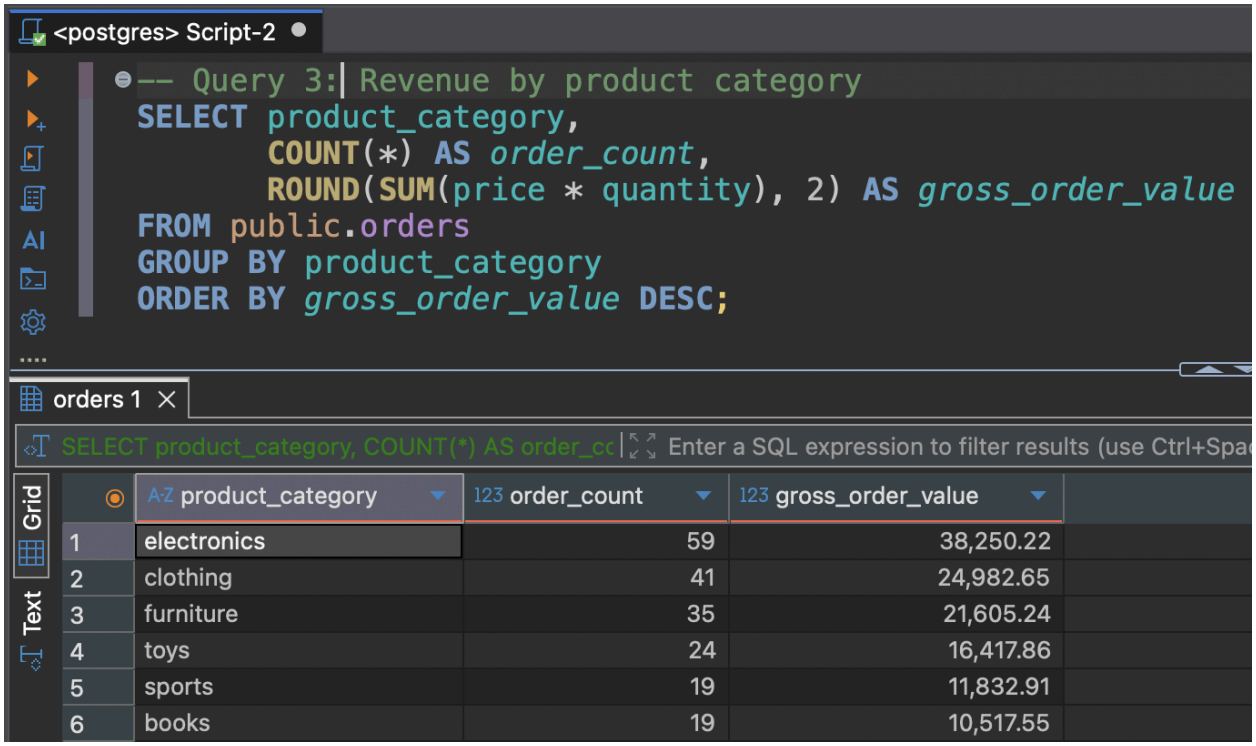
```sql
CREATE TABLE customers (
    customer_id   VARCHAR(20) PRIMARY KEY,
    customer_name VARCHAR(150) NOT NULL,
    email         VARCHAR(254) NOT NULL,
    signup_date   DATE NOT NULL,
    customer_city VARCHAR(100) NOT NULL
)

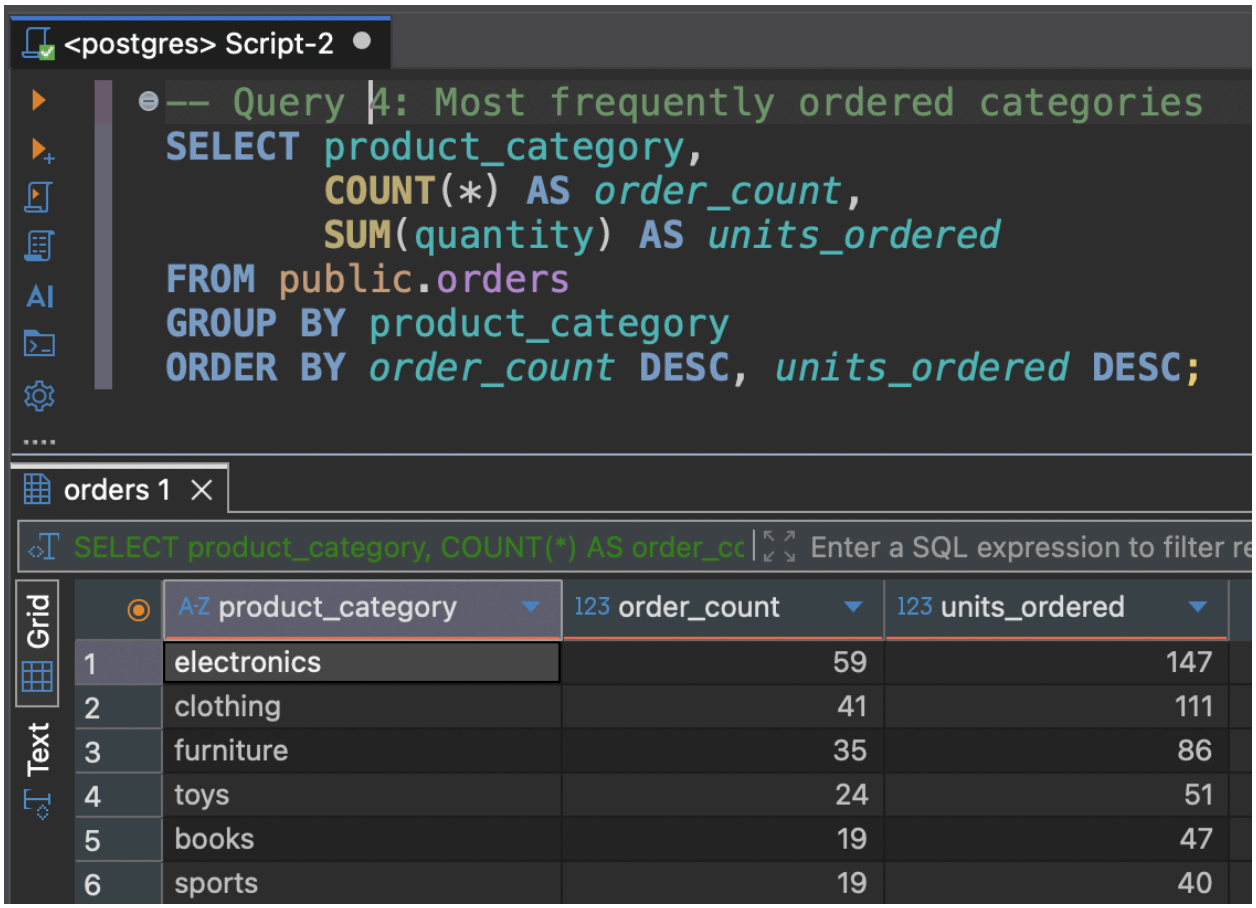
CREATE TABLE orders (
    order_id        VARCHAR(20) PRIMARY KEY,
    customer_id     VARCHAR(20) NOT NULL,
    purchase_date   DATE NOT NULL,
    delivery_date   DATE NULL,
    product_category VARCHAR(50) NOT NULL,
    price           NUMERIC(12, 3) NOT NULL CHECK (price ⩾ 0),
    quantity        INTEGER NOT NULL CHECK (quantity > 0),
    payment_type    VARCHAR(30) NOT NULL,
    customer_city   VARCHAR(100) NOT NULL,
    review_score    NUMERIC(2, 1) NOT NULL CHECK (review_score BETWEEN 1 AND 5),
    review_comment  TEXT NOT NULL
)
```

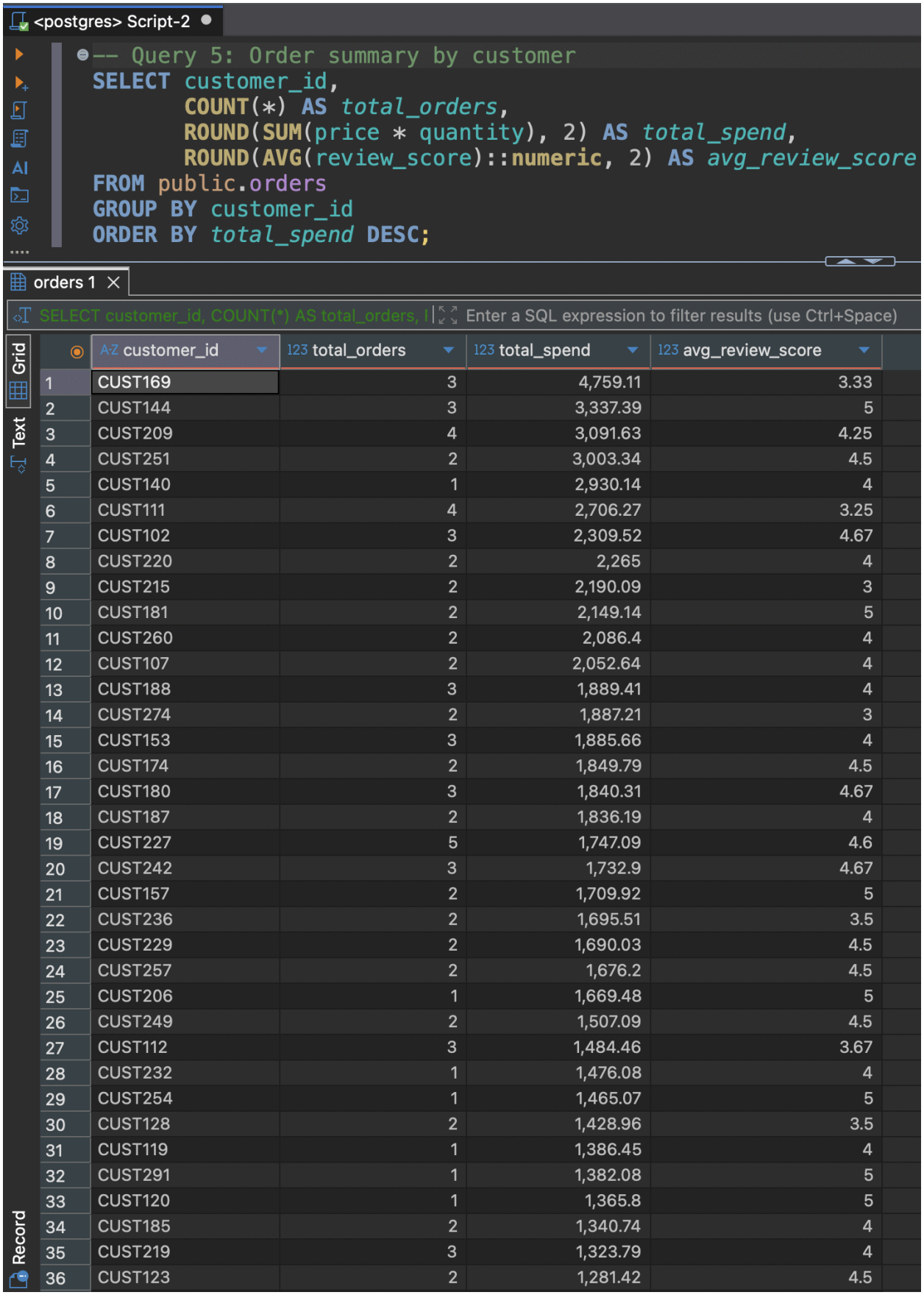

<postgres> Script-2
-- Query 1: Inspect customers
SELECT customer_id, customer_name, customer_city, signup_date
FROM public.customers
ORDER BY signup_date DESC
LIMIT 10;
customers 1
SELECT customer_id, customer_name, customer_
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
customer_id
customer_name
customer_city
signup_date
1 CUST250 Felipe Ferreira belo horizonte 2023-12-01
2 CUST253 Hugo Souza rio de janeiro 2023-11-27
3 CUST208 Carla Almeida rio de janeiro 2023-11-10
4 CUST259 Carla Souza sao paulo 2023-11-08
5 CUST179 Diego Pereira sao paulo 2023-11-07
6 CUST246 Carla Costa curitiba 2023-11-07
7 CUST149 Otavio Souza curitiba 2023-11-03
8 CUST234 Karina Santos belo horizonte 2023-10-30
9 CUST198 Lucas Santos belo horizonte 2023-10-23
10 CUST125 Elena Almeida curitiba 2023-10-22



	order_id	customer_id	product_category	price	quantity	line_total
1	ORD1054	CUST140	electronics	732.535	4	2,930.14
2	ORD1073	CUST259	electronics	732.535	1	732.535
3	ORD1066	CUST254	furniture	732.535	2	1,465.07
4	ORD1194	CUST251	sports	732.535	4	2,930.14
5	ORD1195	CUST209	clothing	497.86	1	497.86
6	ORD1014	CUST210	sports	497.33	1	497.33
7	ORD1161	CUST109	furniture	496.69	1	496.69
8	ORD1062	CUST187	books	495.67	1	495.67
9	ORD1068	CUST121	electronics	495.17	1	495.17
10	ORD1009	CUST229	electronics	488.54	2	977.08
11	ORD1141	CUST104	toys	485.11	2	970.22
12	ORD1115	CUST169	electronics	482.06	4	1,928.24
13	ORD1138	CUST242	toys	478.4	1	478.4
14	ORD1145	CUST102	electronics	478.01	2	956.02
15	ORD1043	CUST293	clothing	470.52	1	470.52
16	ORD1011	CUST181	furniture	463.1	1	463.1
17	ORD1096	CUST119	toys	462.15	3	1,386.45
18	ORD1132	CUST144	sports	454.73	3	1,364.19
19	ORD1122	CUST215	furniture	451.64	4	1,806.56
20	ORD1089	CUST236	electronics	451.01	3	1,353.03
21	ORD1109	CUST285	electronics	444.24	1	444.24
22	ORD1165	CUST169	toys	442.54	4	1,770.16
23	ORD1118	CUST209	furniture	440.14	2	880.28
24	ORD1034	CUST217	books	438.42	1	438.42
25	ORD1170	CUST188	clothing	436.99	3	1,310.97
26	ORD1025	CUST174	books	434.77	3	1,304.31
27	ORD1125	CUST286	electronics	433.64	1	433.64
28	ORD1117	CUST101	toys	432.37	2	864.74
29	ORD1041	CUST242	sports	426.86	2	853.72
30	ORD1013	CUST260	clothing	426.12	3	1,278.36
31	ORD1123	CUST274	electronics	422.31	4	1,689.24
32	ORD1162	CUST181	electronics	421.51	4	1,686.04
33	ORD1030	CUST245	electronics	420.78	2	841.56
34	ORD1188	CUST206	sports	417.37	4	1,669.48
35	ORD1051	CUST107	toys	416.81	4	1,667.24
36	ORD1191	CUST294	clothing	414.7	2	829.4
37	ORD1192	CUST127	electronics	413.92	2	827.84
38	ORD1199	CUST216	furniture	410.32	2	820.64


<postgres> Script-2
-- Query 3: Revenue by product category
SELECT product_category,
COUNT(*) AS order_count,
ROUND(SUM(price * quantity), 2) AS gross_order_value
FROM public.orders
GROUP BY product_category
ORDER BY gross_order_value DESC;
orders 1
SELECT product_category, COUNT(*) AS order_cc
Enter a SQL expression to filter results (use Ctrl+Spa
Grid
Text
product_category
order_count
gross_order_value
1 electronics 59 38,250.22
2 clothing 41 24,982.65
3 furniture 35 21,605.24
4 toys 24 16,417.86
5 sports 19 11,832.91
6 books 19 10,517.55


<postgres> Script-2
-- Query 4: Most frequently ordered categories
SELECT product_category,
COUNT(*) AS order_count,
SUM(quantity) AS units_ordered
FROM public.orders
GROUP BY product_category
ORDER BY order_count DESC, units_ordered DESC;
orders 1
SELECT product_category, COUNT(*) AS order_cc
Enter a SQL expression to filter re
Grid
Text
product_category
order_count
units_ordered
1 electronics 59 147
2 clothing 41 111
3 furniture 35 86
4 toys 24 51
5 books 19 47
6 sports 19 40


<postgres> Script-2
-- Query 5: Order summary by customer
SELECT customer_id,
COUNT(*) AS total_orders,
ROUND(SUM(price * quantity), 2) AS total_spend,
ROUND(AVG(review_score)::numeric, 2) AS avg_review_score
FROM public.orders
GROUP BY customer_id
ORDER BY total_spend DESC;
orders 1
SELECT customer_id, COUNT(*) AS total_orders,
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
Record
customer_id
total_orders
total_spend
avg_review_score
1 CUST169 3 4,759.11 3.33
2 CUST144 3 3,337.39 5
3 CUST209 4 3,091.63 4.25
4 CUST251 2 3,003.34 4.5
5 CUST140 1 2,930.14 4
6 CUST111 4 2,706.27 3.25
7 CUST102 3 2,309.52 4.67
8 CUST220 2 2,265 4
9 CUST215 2 2,190.09 3
10 CUST181 2 2,149.14 5
11 CUST260 2 2,086.4 4
12 CUST107 2 2,052.64 4
13 CUST188 3 1,889.41 4
14 CUST274 2 1,887.21 3
15 CUST153 3 1,885.66 4
16 CUST174 2 1,849.79 4.5
17 CUST180 3 1,840.31 4.67
18 CUST187 2 1,836.19 4
19 CUST227 5 1,747.09 4.6
20 CUST242 3 1,732.9 4.67
21 CUST157 2 1,709.92 5
22 CUST236 2 1,695.51 3.5
23 CUST229 2 1,690.03 4.5
24 CUST257 2 1,676.2 4.5
25 CUST206 1 1,669.48 5
26 CUST249 2 1,507.09 4.5
27 CUST112 3 1,484.46 3.67
28 CUST232 1 1,476.08 4
29 CUST254 1 1,465.07 5
30 CUST128 2 1,428.96 3.5
31 CUST119 1 1,386.45 4
32 CUST291 1 1,382.08 5
33 CUST120 1 1,365.8 5
34 CUST185 2 1,340.74 4
35 CUST219 3 1,323.79 4
36 CUST123 2 1,281.42 4.5

<postgres> Script-2

```sql
-- Query 6: Join orders to customer details. LEFT JOIN preserves orders
-- whose customer_id is absent from the customers file.
SELECT o.order_id, o.customer_id,
       c.customer_name,
       o.product_category,
       o.price, o.quantity,
       ROUND((o.price * o.quantity)::numeric, 2) AS line_total
FROM public.orders o
LEFT JOIN public.customers c ON c.customer_id = o.customer_id
ORDER BY o.purchase_date DESC;
```

orders(+) 1

SELECT o.order_id, o.customer_id, c.customer_na | Enter a SQL expression to filter results (use Ctrl+Space)

Grid | Text

	order_id	customer_id	customer_name	product_category	price	quantity
1	ORD1072	CUST229	Mariana Almeida	books	237.65	3
2	ORD1005	CUST180	Isabela Souza	furniture	185.65	4
3	ORD1079	CUST165	Karina Almeida	electronics	303.37	2
4	ORD1167	CUST200	Nina Almeida	clothing	190.95	3
5	ORD1045	CUST125	Elena Almeida	books	177.84	2
6	ORD1197	CUST111	Joao Pereira	toys	323.13	2
7	ORD1046	CUST219	Joao Ferreira	toys	217.82	2
8	ORD1051	CUST107	[NULL]	toys	416.81	4
9	ORD1042	CUST129	Gabriela Santos	electronics	266.3	3
10	ORD1097	CUST111	Joao Pereira	clothing	257.99	3
11	ORD1049	CUST257	Diego Santos	clothing	286.77	4
12	ORD1135	CUST137	Ana Costa	electronics	339.75	2
13	ORD1080	CUST163	Gabriela Almeida	furniture	184.8	3

<postgres> Script-2

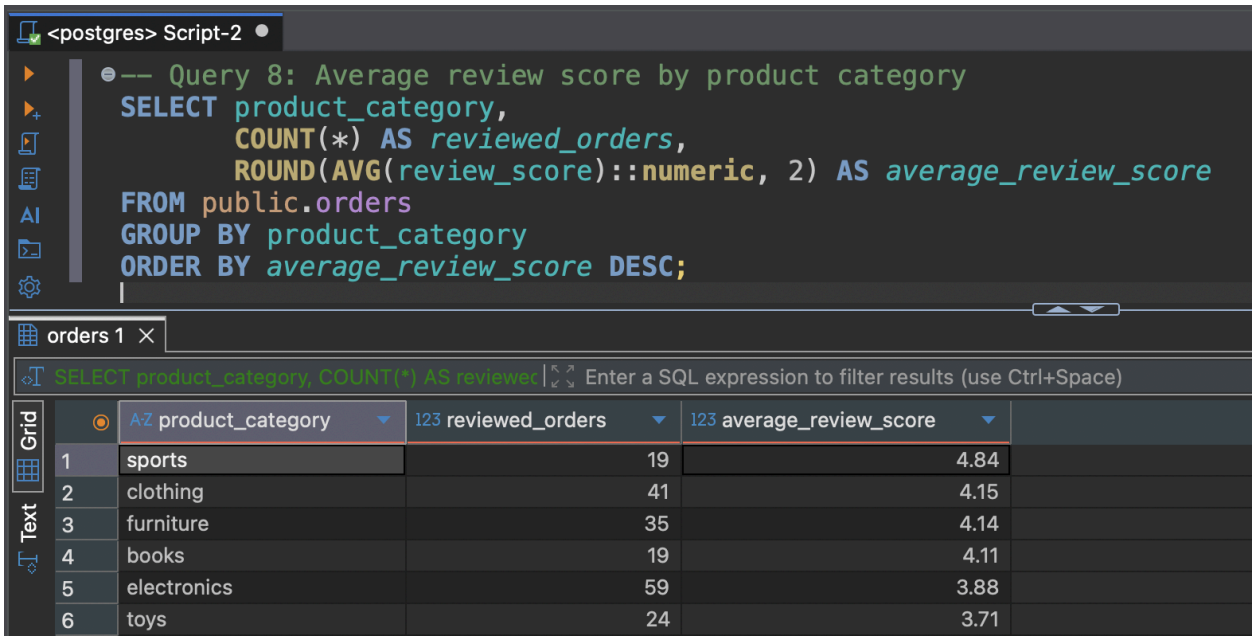
```sql
-- Query 7: Delivery duration in days, excluding missing delivery dates
SELECT order_id, purchase_date, delivery_date,
       delivery_date - purchase_date AS delivery_days
FROM public.orders
WHERE delivery_date IS NOT NULL
ORDER BY delivery_days DESC, order_id;
```

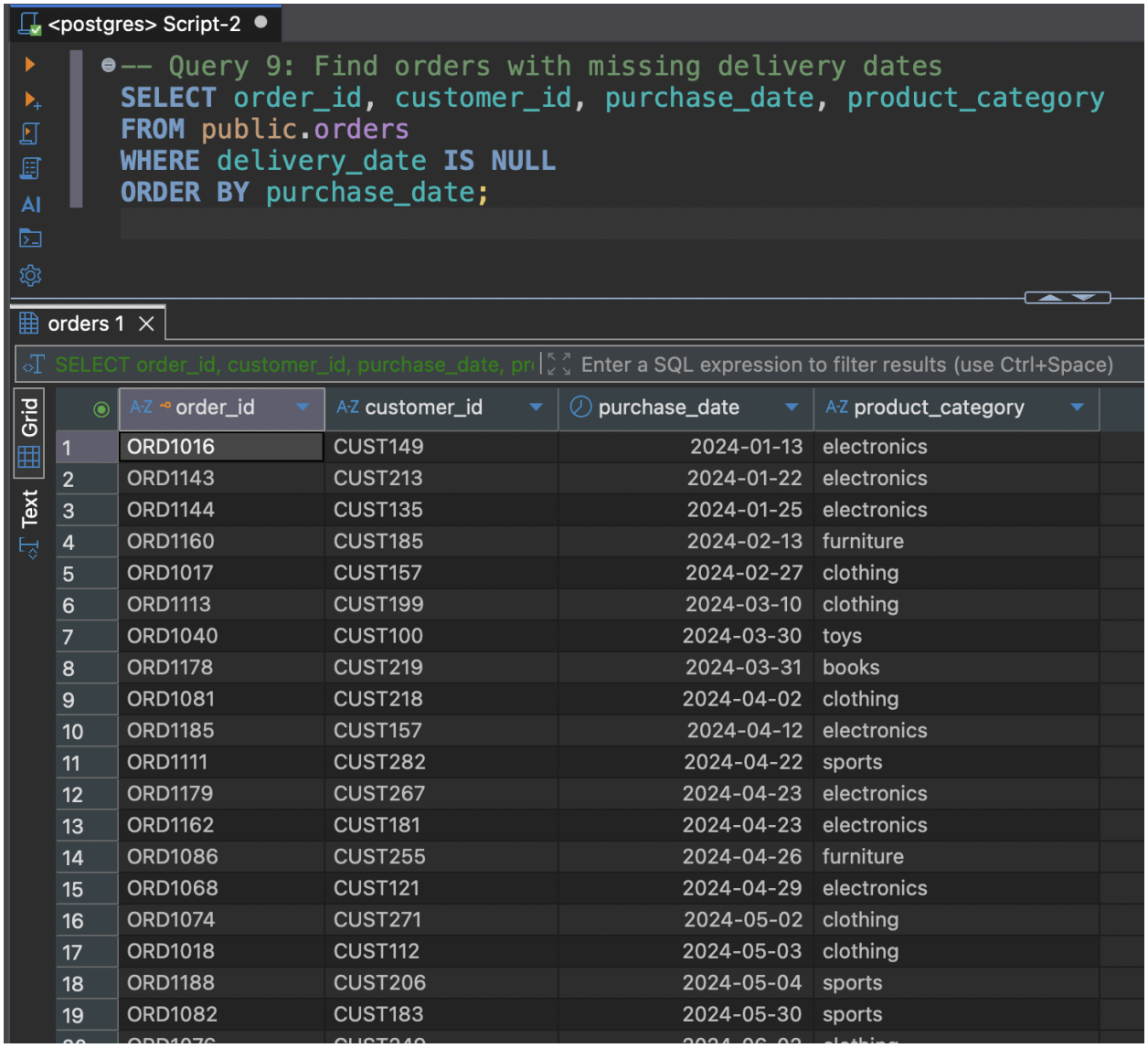
orders 1

SELECT order_id, purchase_date, delivery_date, d | Enter a SQL expression to filter results (use Ctrl+Space)

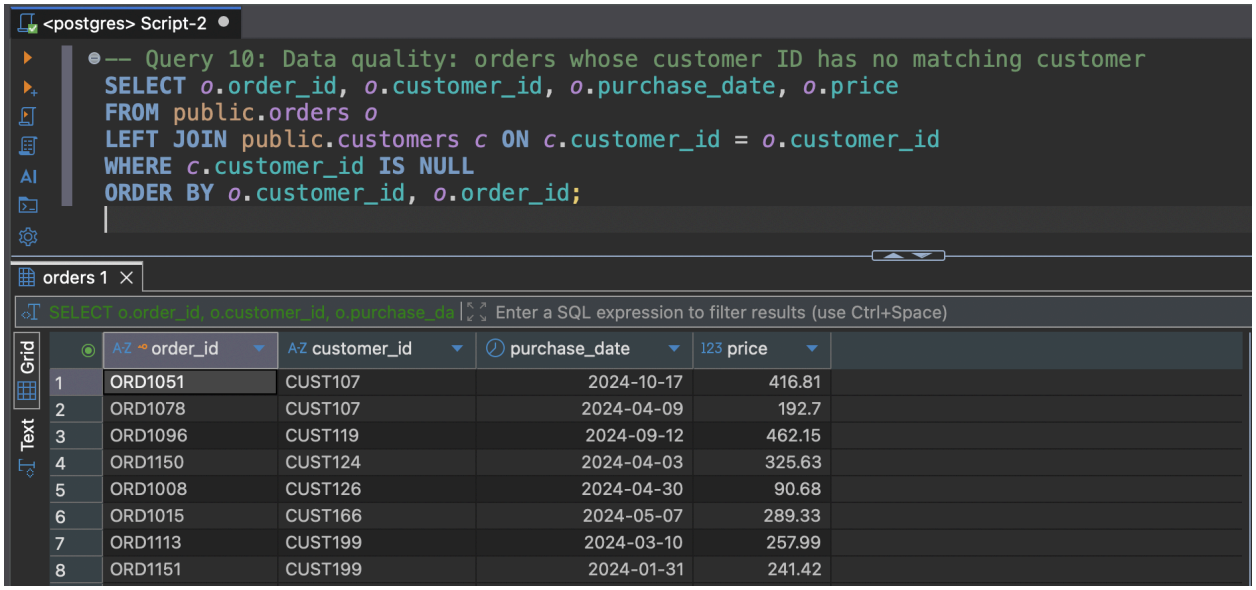
Grid | Text

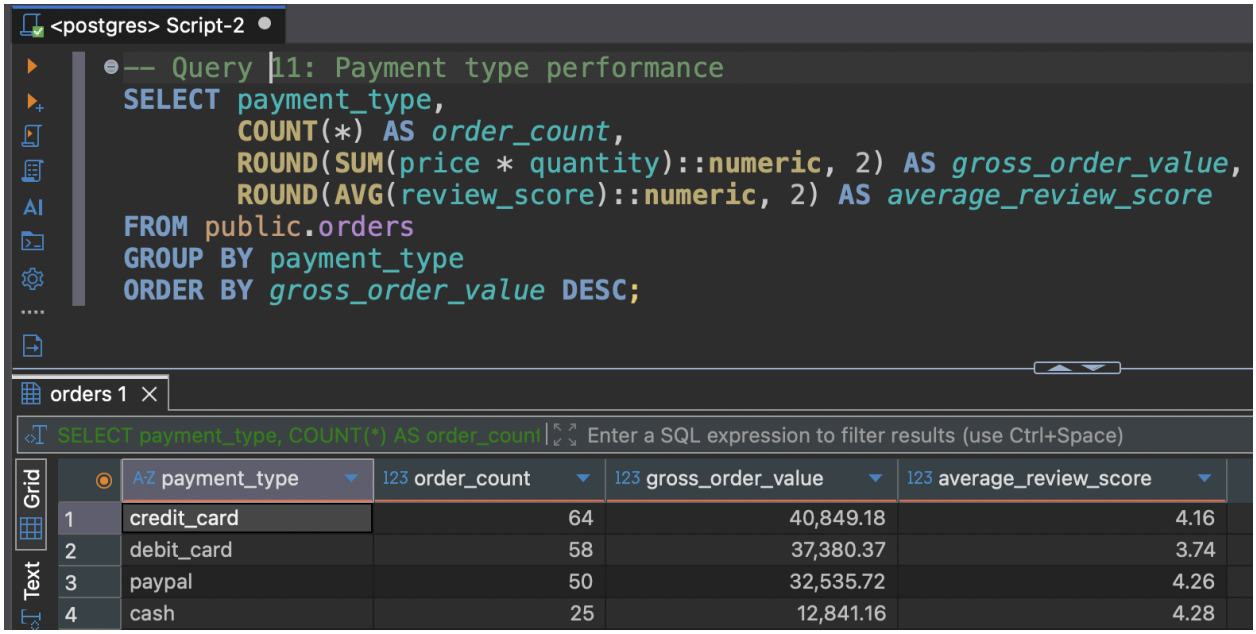
	order_id	purchase_date	delivery_date	delivery_days
1	ORD1034	2024-01-08	2024-11-01	298
2	ORD1132	2024-01-01	2024-09-01	244
3	ORD1138	2024-01-09	2024-09-06	241
4	ORD1015	2024-05-07	2024-10-05	151
5	ORD1009	2024-04-22	2024-09-05	136
6	ORD1093	2024-07-26	2024-11-08	105
7	ORD1187	2024-07-08	2024-08-25	48
8	ORD1104	2024-03-18	2024-05-04	47
9	ORD1039	2024-01-04	2024-02-01	28
10	ORD1007	2024-02-05	2024-02-24	19
11	ORD1026	2024-05-11	2024-05-30	19
12	ORD1066	2024-07-06	2024-07-25	19
13	ORD1085	2024-03-05	2024-03-24	19
14	ORD1119	2024-10-05	2024-10-24	19
15	ORD1139	2024-06-10	2024-06-29	19

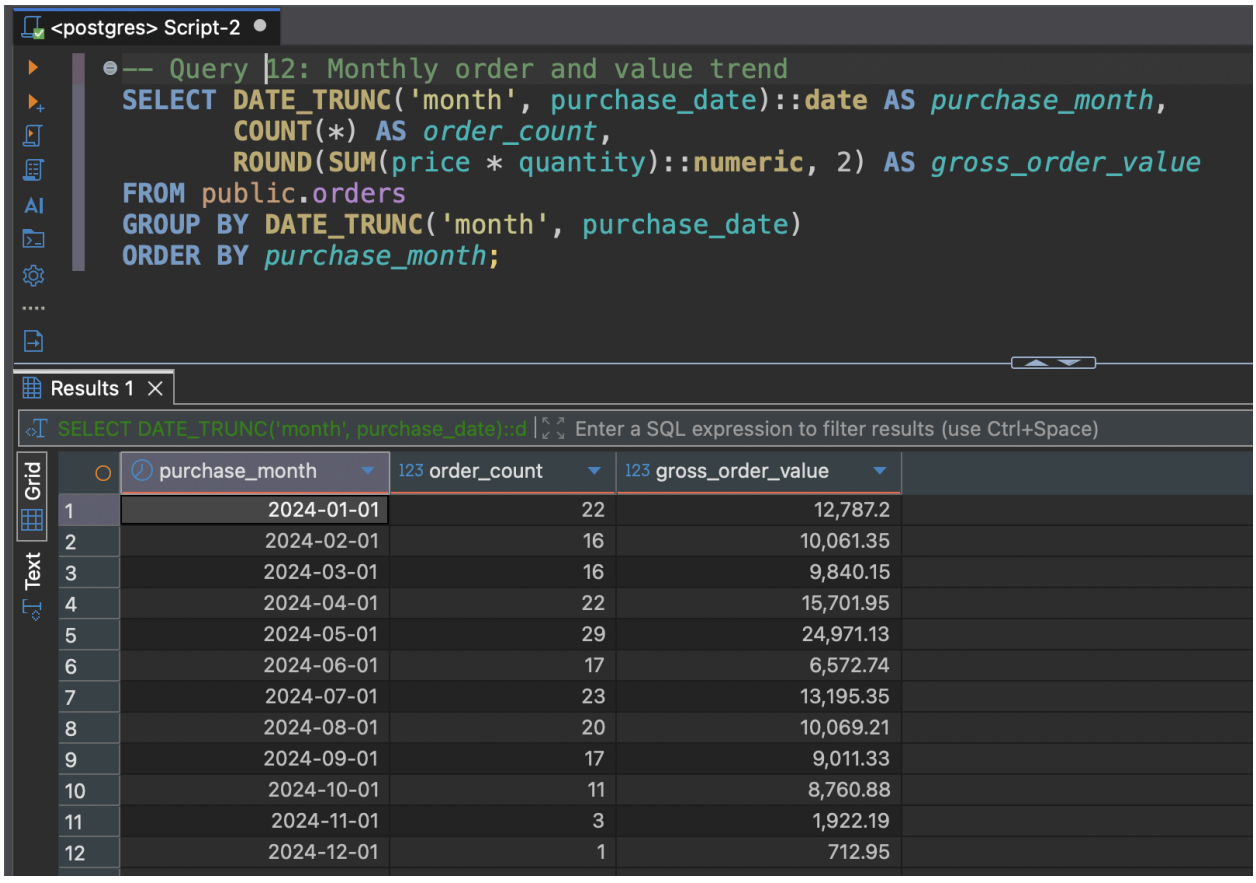

<postgres> Script-2
-- Query 8: Average review score by product category
SELECT product_category,
COUNT(*) AS reviewed_orders,
ROUND(AVG(review_score)::numeric, 2) AS average_review_score
FROM public.orders
GROUP BY product_category
ORDER BY average_review_score DESC;
orders 1
SELECT product_category, COUNT(*) AS reviewed
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
product_category
reviewed_orders
average_review_score
1 sports 19 4.84
2 clothing 41 4.15
3 furniture 35 4.14
4 books 19 4.11
5 electronics 59 3.88
6 toys 24 3.71



	order_id	customer_id	purchase_date	product_category
1	ORD1016	CUST149	2024-01-13	electronics
2	ORD1143	CUST213	2024-01-22	electronics
3	ORD1144	CUST135	2024-01-25	electronics
4	ORD1160	CUST185	2024-02-13	furniture
5	ORD1017	CUST157	2024-02-27	clothing
6	ORD1113	CUST199	2024-03-10	clothing
7	ORD1040	CUST100	2024-03-30	toys
8	ORD1178	CUST219	2024-03-31	books
9	ORD1081	CUST218	2024-04-02	clothing
10	ORD1185	CUST157	2024-04-12	electronics
11	ORD1111	CUST282	2024-04-22	sports
12	ORD1179	CUST267	2024-04-23	electronics
13	ORD1162	CUST181	2024-04-23	electronics
14	ORD1086	CUST255	2024-04-26	furniture
15	ORD1068	CUST121	2024-04-29	electronics
16	ORD1074	CUST271	2024-05-02	clothing
17	ORD1018	CUST112	2024-05-03	clothing
18	ORD1188	CUST206	2024-05-04	sports
19	ORD1082	CUST183	2024-05-30	sports


<postgres> Script-2
-- Query 10: Data quality: orders whose customer ID has no matching customer
SELECT o.order_id, o.customer_id, o.purchase_date, o.price
FROM public.orders o
LEFT JOIN public.customers c ON c.customer_id = o.customer_id
WHERE c.customer_id IS NULL
ORDER BY o.customer_id, o.order_id;
orders 1
SELECT o.order_id, o.customer_id, o.purchase_da
Enter a SQL expression to filter results (use Ctrl+Space)
Grid
Text
order_id
customer_id
purchase_date
price
1 ORD1051 CUST107 2024-10-17 416.81
2 ORD1078 CUST107 2024-04-09 192.7
3 ORD1096 CUST119 2024-09-12 462.15
4 ORD1150 CUST124 2024-04-03 325.63
5 ORD1008 CUST126 2024-04-30 90.68
6 ORD1015 CUST166 2024-05-07 289.33
7 ORD1113 CUST199 2024-03-10 257.99
8 ORD1151 CUST199 2024-01-31 241.42


-- Query 11: Payment type performance
SELECT payment_type,
COUNT(*) AS order_count,
ROUND(SUM(price * quantity)::numeric, 2) AS gross_order_value,
ROUND(AVG(review_score)::numeric, 2) AS average_review_score
FROM public.orders
GROUP BY payment_type
ORDER BY gross_order_value DESC;
orders 1
SELECT payment_type, COUNT(*) AS order_count
Enter a SQL expression to filter results (use Ctrl+Space)
payment_type
order_count
gross_order_value
average_review_score
1 credit_card 64 40,849.18 4.16
2 debit_card 58 37,380.37 3.74
3 paypal 50 32,535.72 4.26
4 cash 25 12,841.16 4.28


-- Query 12: Monthly order and value trend
SELECT DATE_TRUNC('month', purchase_date)::date AS purchase_month,
COUNT(*) AS order_count,
ROUND(SUM(price * quantity)::numeric, 2) AS gross_order_value
FROM public.orders
GROUP BY DATE_TRUNC('month', purchase_date)
ORDER BY purchase_month;
Results 1
SELECT DATE_TRUNC('month', purchase_date)::d
Enter a SQL expression to filter results (use Ctrl+Space)
purchase_month
order_count
gross_order_value
1 2024-01-01 22 12,787.2
2 2024-02-01 16 10,061.35
3 2024-03-01 16 9,840.15
4 2024-04-01 22 15,701.95
5 2024-05-01 29 24,971.13
6 2024-06-01 17 6,572.74
7 2024-07-01 23 13,195.35
8 2024-08-01 20 10,069.21
9 2024-09-01 17 9,011.33
10 2024-10-01 11 8,760.88
11 2024-11-01 3 1,922.19
12 2024-12-01 1 712.95